

Python Fundamentals

Part 2 — Concepts

Conditional Statements · Loops · Functions

1. Conditional Statements

Conditional statements let our program decide what to do based on conditions (i.e. true or false expressions). There are 3 main conditional statements in Python:

Keyword	Purpose
if	Used to test a condition. If it's True, the indented block runs.
elif	("else if") Used when we have multiple conditions to check.
else	Runs if all above conditions are False.

Example 1 — Age Check

```
age = int(input("enter age: "))
if age >= 18:
    print("you can vote")
    print("you can drive")
else:
    print("you can't vote")
    print("you can't drive")
```

Example 2 — Traffic Lights

```
color = input("enter color: ")
if color == "red":
    print("Stop")
elif color == "yellow":
    print("Look")
elif color == "green":
    print("Go")
else:
    print("wrong color")
```

Important Note

We can have standalone if statements and multiple elif statements, but elif and else cannot be used without an if statement preceding them.

Example 3 — Logical Operators in Conditions

Apart from simple conditions, we can also check multiple conditions using logical operators like and, or, and not:

```
age = int(input("enter age: "))
if age < 13:
    print("child")
elif (age >= 13 and age < 18):
    print("teenager")
else:
    print("adult")
```

Nesting in Conditionals

Nesting means placing one block of code inside another. In conditionals, it means putting one if (or if-elif-else) inside another. This helps when you need to make a decision based on another decision.

Login System Example

```
username = input("enter username: ")
password = input("enter password: ")

if (username == "admin" and password == "pass"):
    print("log in successful!")
else:
    if username != "admin":          # NESTING
        print("wrong user name, try again.")
    else:
        print("wrong password, try again.")
```

Ternary Statements

Ternary statements (or conditional expressions) are a compact way of writing if-else in one line.

Syntax: value_if_true if condition else value_if_false

One line decides between 2 values based on a condition:

```
age = 18
status = "Adult" if age >= 18 else "Not Adult"
print(status)    # Adult
```

Note

Everything written with ternary statements can be done with normal if-else. Ternary is best for simple conditions — not recommended for nested or complex conditions.

Match-Case Statements

In Python, match / case statements are an alternative to long chains of if-elif-else. They were introduced in Python 3.10.

Syntax:

```
match variable:
    case pattern1:
        # code if pattern1 matches
    case pattern2:
        # code if pattern2 matches
    case _:
        # default case (like else)
```

1. **match** — what you're comparing
2. **case** — the value or pattern to match against
3. **case _** — wildcard (matches anything, like "default")

Traffic Lights with match-case

```
color = input("enter color: ")
match color:
    case "red":
        print("Stop")
    case "yellow":
        print("Look")
    case "green":
        print("Go")
    case _:
        print("wrong color")
```

Practice Problems (Set 1)

1. For a given number n , print if it's a multiple of 5 or not.
2. For a given number n , print if it's Odd or Even.

2. Loops

A loop lets us execute a block of code multiple times — either a set number of times or until a condition is met. Python has 2 main types of loops:

Loop Type	Description
while	Repeat while a condition is True
for	Iterate over a sequence (list, string, range, etc.)

2.1 The while Loop

Syntax:

```
while condition:
    # code block
```

All statements in the code block are executed one after the other, as long as the condition remains True.

Example 1 — Print 1 to 5

```
i = 1
while i <= 5:
    print(i)
    i += 1
# output: 1, 2, 3, 4, 5
```

Example 2 — Print 5 to 1

```
i = 5
while i > 0:
    print(i)
    i -= 1
# output: 5, 4, 3, 2, 1
```

Warning — Infinite Loops

If the loop condition always evaluates to True, the loop never stops. This is called an infinite loop. Avoid creating them unintentionally.

```
while True:
    # DO NOT RUN this code
    print("Prime")
```

2.2 Loop Control Statements

Python gives us tools to control how loops behave. The two main ones are `break` and `continue`.

`break` — Stop the loop immediately

```
# Break when multiple of 6 is found
i = 1
while i <= 10:
    if i % 6 == 0:
        break
    print(i)
    i += 1
# output: 1, 2, 3, 4, 5
```

`continue` — Skip current iteration

Skips the rest of the current iteration and moves to the next one.

```
# Skip multiples of 3
i = 0
while i < 10:
    i += 1
    if i % 3 == 0:
        continue
    print(i)
# output: 1, 2, 4, 5, 7, 8, 10
```

2.3 The for Loop

A for loop is used to iterate over a sequence (list, tuple, string, or range) and execute a block of code for each item.

Syntax:

```
for variable in sequence:
    # code to run for each item
for i in range(5):
    print(i)
# output: 0, 1, 2, 3, 4
```

The in Keyword

in is a membership operator in Python. It is used to loop over a sequence or to check if an item exists in a sequence.

Example 1 — Looping over a string

```
word = "Prime"
for ch in word:
    print(ch)
```

Example 2 — Check if character exists

```
word = "Prime"
if 'i' in word:
    print("letter exists")
```

Example 3 — Count occurrences of 'i'

```
word = "artificial intelligence"
count = 0
for ch in word:
    if ch == 'i':
        count += 1
print(f"i occurs {count} times.")
```

Nested Loops

Just like conditionals, we can have nested loops — a loop inside another loop.

```
for i in range(1, 3):
    for j in range(1, 3):
        print(f"({i}, {j})")
# output: (1, 1) (1, 2) (2, 1) (2, 2)
```

2.4 The range() Function

range() generates a sequence of numbers, typically for use in loops. It creates an efficient iterator that produces numbers one by one (it does not create a full list in memory unless converted).

It has 3 parameters:

1. **start** (default = 0) — the number to start from
2. **stop** — the number to stop before (not included)
3. **step** (default = 1) — how much to increase by each time

Single argument — stop only

```
for i in range(5):  
    print(i)  
# output: 0, 1, 2, 3, 4
```

Two arguments — start, stop

```
for i in range(1, 6):  
    print(i)  
# output: 1, 2, 3, 4, 5
```

Three arguments — start, stop, step

```
for i in range(1, 10, 2):  
    print(i)  
# output: 1, 3, 5, 7, 9
```

Practice Problems (Set 2)

1. Print multiplication table for any number n. [using while]
2. Print odd numbers from 1 to 10, using continue. [using while]
3. Count vowels in a word. [using for]
4. Sum of first n natural numbers. [using for]

3. Functions

Functions in Python are blocks of reusable code that perform a specific task. They make our code organized, readable, and easier to maintain.

3.1 Function Definition

We use the `def` keyword to define a function:

```
# Function Definition  
def hello():  
    print("hello from Prime")  
  
# Function Call  
hello()    # invokes the function
```

A function call invokes the function, and then the entire block of code inside the function definition is executed.

3.2 Functions with Parameters

We can pass data to a function using parameters (inside the parentheses). The actual values passed when calling the function are called arguments.

```
# Function to compute sum of 2 numbers  
def sum(a, b):    # a & b are parameters  
    print(a + b)  
  
sum(5, 10)        # 5 & 10 are arguments → 15
```

3.3 Return Values

Functions can return a result using the `return` keyword.

```
# Function to compute average of 3 numbers  
def avg(a, b, c):  
    return (a + b + c) / 3  
  
print(avg(1, 2, 3))    # → 2.0
```

3.4 Default Parameters

We can assign default values to function parameters so that if the caller doesn't provide that argument, Python uses the default. Default parameters must come last — we cannot have a non-default argument after a default argument.

```
def sum(a, b=1):      # default value of b is 1
    return a + b

print(sum(5))         # output: 6
print(sum(5, 3))      # output: 8
```

3.5 Built-in Functions

All functions we define ourselves are user-defined. Python also provides many built-in functions whose logic is already written — we just use them. Some we've already used:

1. `print()`
2. `input()`
3. `type()`
4. `range()`

3.6 Lambda Functions

Lambda functions are short, one-liner functions used for simple tasks. They are anonymous (no name) and created with the `lambda` keyword.

Syntax: `lambda arguments: expression`

The expression result is automatically returned.

```
# Function to compute x2
square = lambda x: x * x
print(square(5))    # → 25
```

Practice Problems (Set 3)

1. Create a function to compute factorial of a number `n`.
2. Create a function to return the largest of 3 numbers — `a`, `b` & `c`.

4. Solutions

Solutions — Set 1 (Conditionals)

1. Multiple of 5 or not

```
num = int(input("enter number: "))
if num % 5 == 0:
    print("multiple of 5")
else:
    print("NOT a multiple of 5")
```

2. Odd or Even

```
num = int(input("enter number: "))
if num % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Solutions — Set 2 (Loops)

1. Multiplication table of `n`

```
n = int(input("enter n: "))
i = 1
while i <= 10:
    print(i * n)
    i += 1
```

2. Odd numbers from 1 to 10 (using continue)

```
i = 0
while i < 10:
    i += 1
    if i % 2 == 0:
        continue
    print(i)
# output: 1, 3, 5, 7, 9
```

3. Count vowels in a word

```
word = input("enter word: ")
count = 0
for ch in word:
    if ch in 'aeiouAEIOU':
        count += 1
print(f"vowel count = {count}")
```

4. Sum of first n natural numbers

```
n = int(input("enter n: "))
total = 0
for i in range(1, n + 1):
    total += i
print("sum =", total)
```

Solutions — Set 3 (Functions)

1. Factorial of n

```
n = int(input("enter n: "))
fact = 1
for i in range(1, n + 1):
    fact *= i
print("factorial =", fact)
```

As a function

You can also wrap this in a function: `def factorial(n): ... return fact`

2. Largest of 3 numbers

```
def get_largest(a, b, c):
    if a > b and a > c:
        return a
    elif b > c:
        return b
    else:
        return c

print(get_largest(3, 10, 5))    # → 10
```

Keep Learning & Keep Exploring!